

Flex

Smart Contract Security Assessment

August 05, 2026



ABSTRACT

Dedaub was commissioned to perform a security audit of the Flex protocol v1.1/v2 upgrade. This upgrade reworks borrow and leverage flows around authenticated callbacks and direct auction settlement, redirects upfront fees to the protocol as first-loss coverage, and adds a configurable cooldown of up to one hour before newly opened or increased debt can be repaid or closed.

BACKGROUND

Flex is a CDP-style lending protocol. Borrowers lock collateral in troves and receive borrow tokens, paying a fixed annual interest rate of their own choosing (bounded, adjustable with a cooldown). Lenders deposit borrow tokens into a Yearn V3 strategy, earning interest and upfront-borrow fees; they also absorb bad debt from underwater liquidations and receive surplus auction proceeds from redemptions. When idle lender liquidity is insufficient – for borrows or for lender withdrawals – the protocol redeems troves and Dutch-auctions their collateral to free up borrow tokens. For borrows, the redeemed troves have strictly lower interest rates than the new borrow, while for lender withdrawals troves are redeemed as necessary, in both cases generally starting from the lowest interest rate trove. Troves can be liquidated by anyone once their collateralization ratio (CR) falls below the minimum (MCR), with a fee that scales linearly between a preset bound based on how far below MCR the trove sits.

SETTING & CAVEATS

This audit report mainly covers the changes introduced to the contracts of the Flex protocol by PR [#20](#) at commit [92dfb87b9a5aeeec994a78a339b76d70ca811cd30](#).

This report builds on the [prior audit reports](#) available in the repository. Previously reported issues that the protocol team has already acknowledged or explicitly decided not to fix are generally not repeated in this report, unless the current review identified a materially

new exploit path, an incomplete mitigation, or a distinct impact not covered by the earlier report. The absence of a previously known and acknowledged issue in this report should not be interpreted as a change in its status or as a reassessment of its validity.

Audit Start Date: Jul 30, 2026

Report Submission Date: Aug 05, 2026

Three auditors worked on the following contracts:

```
flex-contracts/src/  
├── auction.vy  
├── factory.vy  
├── lender/  
│   ├── LenderFactory.sol  
│   └── Lender.sol  
├── periphery/  
│   └── leverage_zapper.vy  
└── trove_manager.vy
```

The audit's main target is security threats, i.e., what the community understanding would likely call "hacking", rather than the regular use of the protocol. Functional correctness (i.e. issues in "regular use") is a secondary consideration. Typically it can only be covered if we are provided with unambiguous (i.e. full-detail) specifications of what is the expected, correct behavior. In terms of functional correctness, we often trusted the code's calculations and interactions, in the absence of any other specification. Functional correctness relative to low-level calculations (including units, scaling and quantities returned from external protocols) is generally most effectively done through thorough testing rather than human auditing.

VULNERABILITIES & FUNCTIONAL ISSUES

This section details issues affecting the functionality of the contract. Dedaub generally categorizes issues according to the following severities, but may also take other considerations into account such as impact or difficulty in exploitation:

Category	Description
CRITICAL	Can be profitably exploited by any knowledgeable third-party attacker to drain a portion of the system's or users' funds OR the contract does not function as intended and severe loss of funds may result.
HIGH	Third-party attackers or faulty functionality may block the system or cause the system or users to lose funds. Important system invariants can be violated.
MEDIUM	Examples: • User or system funds can be lost if external systems misbehave. • DoS, under specific conditions. • Part of the functionality becomes unusable due to a programming error.
LOW	Examples: • Breaking important invariants but without apparent consequences. • Buggy functionality for trusted users where a workaround exists. • Security issues which may manifest when the system evolves.

Issue resolution statuses include “resolved,” when auditors verify that the issue has been addressed; “mitigated,” when residual risk remains but is materially reduced; “acknowledged,” when the client accepts the observation without action; “by design,” when the behavior is intentional; and “dismissed,” when the client does not consider it an issue.

CRITICAL SEVERITY:

[No critical severity issues]

HIGH SEVERITY:

[No high severity issues]

MEDIUM SEVERITY:

[No medium severity issues]

LOW SEVERITY:

[No low severity issues]

OTHER / ADVISORY ISSUES:

This section details issues that are not thought to directly affect the functionality of the project, but we recommend considering them.

ID	Description	STATUS
A1	Equivalent helper-Trove unwinds can bypass the repayment cooldown and suppress upfront fees	DISMISSED
The upfront fee charged by <code>open_trove()</code>, <code>borrow()</code>, and premature <code>adjust_interest_rate()</code> operations is calculated from the current aggregate debt-weighted interest rate. This basis includes debt introduced by the operation being quoted as well as all existing Troves. <code>src/trove_manager.vy::_get_upfront_fee():1517-1527</code> <pre> <i>## Total debt after adding the new debt</i> new_total_debt: uint256 = self.total_debt if is_existing_debt else self.total_debt + debt_amount <i>## Total weighted debt after adding the new weighted debt</i> new_total_weighted_debt: uint256 = self.total_weighted_debt if is_existing_debt else self.total_weighted_debt + (debt_amount * annual_interest_rate) <i>## Calculate the new average interest rate</i> avg_interest_rate: uint256 = new_total_weighted_debt // new_total_debt <i>## Calculate the upfront fee using the average interest rate</i> upfront_fee: uint256 = self._calculate_accrued_interest(debt_amount * avg_interest_rate, self.upfront_interest_period) </pre>		

```
Make sure the user is ok with the upfront fee  
assert upfront_fee <= max_upfront_fee, "!max_upfront_fee"
```

An attacker can introduce a large helper Trove at the minimum interest rate before performing a genuine higher-rate action. The helper lowers `total_weighted_debt / total_debt`, causing the genuine action to permanently capitalize a smaller upfront fee.

The protocol attempts to prevent temporary helper debt by recording `last_debt_increase_time` and requiring the repayment cooldown to expire before direct repayment or closure.

`src/trove_manager.vy::repay():738-739`

```
Enforce the cooldown after the last debt increase, so debt cannot be  
created and repaid atomically  
assert block.timestamp > convert(trove.last_debt_increase_time, uint256) +  
    self.repay_cooldown, "!repay_cooldown"
```

However, the cooldown is enforced only by the owner-facing repayment and closure functions. It is not enforced when `_redeem()` removes debt from a Trove. Redemption updates `last_debt_update_time`, but leaves `last_debt_increase_time` unchanged.

`src/trove_manager.vy::_redeem():1391-1397`

```
Update the Trove's info  
trove.debt = trove_new_debt  
trove.collateral = trove_new_collateral  
trove.last_debt_update_time = convert(block.timestamp, uint64)
```

```
## Save changes to storage  
self.troves[trove_to_redeem] = trove
```

This enables the following sequence:

1. The attacker supplies or relies on sufficient Lender liquidity and opens a large helper Trove at the minimum interest rate.
2. The helper consumes the available Lender liquidity and immediately enters the aggregate fee basis.
3. The attacker opens a genuine Trove at a higher rate. Its upfront fee is calculated against the helper-depressed average.
4. Because Lender idle liquidity has been consumed, delivery of the genuine principal invokes `_redeem()`.
5. Rate ordering selects the attacker's low-rate helper and removes most of its debt without checking the repayment cooldown.
6. The resulting Auction is settled through the permitted callback flow. The redeemed collateral is used as part of the genuine Trove's collateral.
7. Redemption leaves the helper with a small active residual debt. The attacker calls `remove_collateral()` and releases all collateral above the residual Trove's minimum collateral ratio.

`remove_collateral()` checks the resulting collateral ratio, but does not check whether the collateral became excess through the redemption of recently created debt.

`src/trove_manager.vy::remove_collateral():578-594`

```
## Calculate the new collateral amount and collateral ratio  
new_collateral: uint256 = trove.collateral - collateral_amount  
collateral_ratio: uint256 = self._calculate_collateral_ratio(
```



```
        new_collateral, trove_debt_after_interest, collateral_price
    )

    # Make sure the new collateral ratio is above the minimum collateral ratio
    assert collateral_ratio >= self.minimum_collateral_ratio,
        "!minimum_collateral_ratio"

    # Update the Trove's collateral info
    self.troves[trouve_id].collateral = new_collateral

    # Update the contract's recorded collateral balance
    self.collateral_balance -= collateral_amount

    # Transfer the collateral tokens to caller
    assert extcall self.collateral_token.transfer(
        msg.sender, collateral_amount, default_return_value=True
    )
```

A focused upper-bound validation used the maximum permitted one-hour repayment cooldown, a one-million-USDC helper at the minimum 0.1% interest rate, an approximately one-million-USDC genuine Trove at the maximum 250% interest rate, and a seven-day upfront-interest period. It produced the following result:

Honest genuine upfront fee:	47,898.179771 USDC
Helper upfront fee:	19.178082 USDC
Manipulated genuine upfront fee:	23,946.675817 USDC
Protocol-fee undercollection:	23,932.325872 USDC

The validation removes most of the helper debt and releases its excess collateral in the helper's debt-increase timestamp, while a direct one-unit repayment still reverts with `!repay_cooldown`.

The validation represents a newly deployed or effectively empty market. Existing system debt dilutes the helper's influence on the aggregate rate and may increase the helper's own upfront fee. In a mature market, the attacker must scale the helper relative to the existing debt base. Profitability depends on both the amount and average rate of pre-existing debt. The result is measured before gas, collateral opportunity cost, price risk, and execution friction.

Related validations also demonstrated profitable aged-helper and interest-rate-adjustment variants. In particular, `adjust_interest_rate()` can lower a pre-aged helper's weighted-debt contribution without refreshing `last_debt_increase_time`. The helper can then influence a genuine fee quote and close in the same timestamp. These variants reinforce that a timestamp applied only to named repayment functions does not make the branch-average fee basis manipulation-resistant.

The direct loss is the reduction of `unclaimed_protocol_fees`, which lowers protocol revenue and the first-loss reserve available before bad debt reaches Lender shareholders. As a secondary effect, the omitted fee is never added to the target Trove's debt and weighted debt, so the Lender also forgoes the future interest that would have accrued on that amount. The demonstrated path does not by itself create immediate insolvency or remove existing Lender principal.

The validated sequence does not rely on closing a zombie Trove. The redemption is sized to leave the helper active with residual debt above `min_debt`, after which the attacker can remove the collateral exceeding the residual Trove's minimum collateral requirement. However, the validation used attacker-owned collateral and did not establish a production flash-loan-funded route. At a 110% minimum collateral ratio, the one-million-USDC helper requires approximately 1.1 million USDC of collateral value to be available when it is opened, in addition to the equity top-up required by the genuine position. Most of this gross funding requirement is not a permanent

manipulation cost. The helper delivers one million USDC to the attacker, the redeemed collateral is recycled into the genuine Trove, and the helper's excess collateral is released immediately. The borrowed principal remains paired with the final outstanding debt and is not attacker profit. Even so, the path is capital-intensive and its practical profitability depends on Lender liquidity, the existing debt base, redemption ordering, rate spread, gas, and collateral acquisition costs.

Comments:

The team stated that it does not consider the scenario practically exploitable because a flash-loan-funded attacker cannot close the helper after it becomes a zombie during the repayment cooldown.

A2	Same-ID re-kicks can execute stale take calldata at a higher payment	DISMISSED
----	--	-----------

An Auction can be re-kicked under the same `auction_id` after its current price epoch becomes inactive. Re-kicking replaces the timestamp and price parameters without changing the identifier used by pending `take()` transactions.

The public Dutch Desk entry point has no caller restriction. It calculates new prices and forwards them to the existing Auction ID.

`src/dutch_desk.vy::re_kick():153-178`

```
@external
def re_kick(auction_id: uint256):
    """
    ...
    """

    # Cache the Auction contract
    auction: IAuction = self.auction
```

```
## Get the auction info
auction_info: IAuction.AuctionInfo = staticcall
    auction.auctions(auction_id)

## Get new starting and minimum prices
starting_price: uint256 = 0
minimum_price: uint256 = 0
starting_price, minimum_price = self._get_prices(
    auction_info.current_amount,
    self.re_kick_starting_price_buffer_percentage,
)

## Re-kick with new prices
extcall auction.re_kick(auction_id, starting_price, minimum_price)
```

The Auction updates the existing record's `kick_timestamp`, `initial_amount`, `starting_price`, and `minimum_price`.

`src/auction.vy::re_kick():331-347`

```
## Make sure minimum price is non-zero
assert minimum_price != 0, "!minimum_price"

## Bring auction info into memory
auction: AuctionInfo = self.auctions[auction_id]

## Make sure auction is not already active
assert not self._is_active(auction), "active"

## Make sure there's actually something to kick
assert auction.current_amount != 0, "!current_amount"
```

```
# Update kick timestamp, initial amount, starting price, and minimum price
auction.kick_timestamp = block.timestamp
auction.initial_amount = auction.current_amount
auction.starting_price = starting_price
auction.minimum_price = minimum_price
```

`take()` lets the caller specify the Auction ID and maximum collateral quantity, but does not accept a maximum buy-token payment, deadline, expected kick timestamp, or expected epoch. The function recalculates `needed_amount` from whichever epoch is stored when the transaction executes.

`src/auction.vy::take():358-389`

```
@external
def take(
    auction_id: uint256,
    max_take_amount: uint256 = max_value(uint256),
    receiver: address = msg.sender,
    data: Bytes[_MAX_CALLBACK_DATA_SIZE] =
        empty(Bytes[_MAX_CALLBACK_DATA_SIZE]),
) -> uint256:
    """
    ...
    """

    # Bring auction info into memory
    auction: AuctionInfo = self.auctions[auction_id]

    # Make sure auction is active
    assert self._is_active(auction), "!active"
```

```
# Determine amount to take
take_amount: uint256 = min(auction.current_amount, max_take_amount)

# Get the needed amount of buy token to pay
needed_amount: uint256 = self._get_amount_needed(auction, take_amount)

# Make sure needed amount is not zero
assert needed_amount != 0, "!needed_amount"
```

The recalculated amount is then transferred to the fixed receiver and, where applicable, the Lender as surplus.

[src/auction.vy::take\(\):418-437](#)

```
# How much of the receiver's capped claim remains unsatisfied
receiver_remaining: uint256 = auction.maximum_amount -
    auction.amount_received

# Consume the receiver's claim first, then transfer any surplus to the
surplus receiver.
# A self-taker consumes its own claim notionally: it receives collateral
in kind, so transferring
# the receiver portion of the buy token to itself would have no economic
effect.
if needed_amount <= receiver_remaining:
    # Entire amount satisfies the receiver's claim
    auction.amount_received += needed_amount
    if msg.sender != auction.receiver:
        assert extcall buy_token.transferFrom(msg.sender,
            auction.receiver, needed_amount, default_return_value=True)
else:
```

```
# Cover the receiver first
if receiver_remaining > 0:
    auction.amount_received = auction.maximum_amount
    if msg.sender != auction.receiver:
        assert extcall buy_token.transferFrom(
            msg.sender,
            auction.receiver,
            receiver_remaining,
            default_return_value=True
        )

# Transfer the surplus to the surplus receiver
surplus: uint256 = needed_amount - receiver_remaining
assert extcall buy_token.transferFrom(msg.sender,
    auction.surplus_receiver, surplus, default_return_value=True)
```

A user can prepare a `take()` transaction from an old active epoch based on the then-current quote. If the transaction remains pending while that epoch expires, another actor can re-kick the Auction before the user's transaction is included. The same calldata remains valid because the Auction ID and collateral amount have not changed, but payment is recalculated from the fresh epoch.

A focused validation preserved the same Auction ID, `max_take_amount`, receiver, and callback data across expiry and a permissionless re-kick using the production 100.1% re-kick buffer:

Old final-active-epoch quote:	22,054.443144 USDC
Post-re-kick debit:	22,298.581831 USDC
Debit above the old quote:	244.138687 USDC
Relative debit increase:	1.10698%

The taker received the same collateral amount but paid approximately 244.138687 USDC more than the prepared quote. This complete amount is quote deterioration, not necessarily mark-to-market loss. In the unchanged-oracle test state, approximately 22.276306 USDC represented a premium over oracle value. The remainder represented the disappearance of the old epoch's expected discount.

A taker that holds or approves only the amount required by the old quote is protected because the post-re-kick `transferFrom()` reverts atomically when it cannot collect the fresh `needed_amount`. A receiver callback that compares the fresh `needed_amount` against the old quote can also revert the transaction. These are useful caller-side protections, but none is enforced by the Auction interface. The focused validation confirms that the same calldata executes at the higher payment when the taker has sufficient buy-token balance and allowance. Callers that maintain larger balances and broad allowances, or that use empty callback data without an in-call payment check, remain exposed.

The distribution of the additional payment depends on the Auction's remaining receiver cap. It may satisfy the fixed receiver's remaining claim or become Lender surplus. This does not affect the taker's total debit.

The issue does not affect the canonical Zapper's fresh self-take in a successful open or lever-up call because no separate re-kick transaction can interleave in that call tree. It applies to later pending Auction takes that cross an expiry and re-kick boundary.

The issue requires specific transaction timing, sufficient buy-token balance and allowance for the fresh payment, a missing callback-side payment check, and an Auction whose fresh epoch is more expensive than the stale quote. The taker still receives the requested collateral, and a limited balance, exact allowance, or checking callback causes the transaction to revert atomically.

Comments:

The team noted the observation but stated that it does not consider the scenario practically relevant because it expects the taker not to hold enough buy tokens to pay the higher post-re-kick price.

DISCLAIMER

The audited contracts have been analyzed using automated techniques and extensive human inspection in accordance with state-of-the-art practices as of the date of this report. The audit makes no statements or warranties on the security of the code. On its own, it cannot be considered a sufficient assessment of the correctness of the contract. While we have conducted an analysis to the best of our ability, it is our recommendation for high-value contracts to commission several independent audits, a public bug bounty program, as well as continuous security auditing and monitoring through Dedaub Security Suite.

ABOUT DEDAUB

Dedaub offers significant security expertise combined with cutting-edge program analysis technology to secure some of the most prominent protocols in DeFi. The founders, as well as many of Dedaub's auditors, have a strong academic research background together with a real-world hacker mentality to secure code. Protocol blockchain developers hire us for our foundational analysis tools and deep expertise in program analysis, reverse engineering, DeFi exploits, cryptography and financial mathematics.